Backend Architecture Proposal: Agentic AI Service Orchestrator

To: Senior Engineering Team

Project Deadline: May 20, 2026

Subject: Backend Infrastructure & Hybrid AI Knowledge Engine Architecture

1. Executive Summary

This proposal outlines the backend architecture for the AI Service Orchestrator. To meet production requirements by our May 20 deadline, we will move away from monolithic LLM wrappers and implement a decentralized, multi-agent system. The backend will utilize Google Agent Development Kit (ADK V2) for orchestration, a Hybrid Tiered AI Knowledge Engine (combining Prompting, Fine-Tuning, CAG, and Agentic RAG) powered by Gemma 4 for reasoning, and a robust state management strategy.

2. Data Foundation & Mock Dataset

The system's logic and testing will be grounded in a comprehensive, cleaned dataset of 50,000 records to accurately simulate the informal economy.

- **Geographic Scope:** The database covers key Pakistani markets: Karachi, Lahore, Islamabad, Rawalpindi, Faisalabad, and Peshawar.
- **Service Categories:** Includes Plumber, Electrician, AC Technician, Beautician, Tutor, Carpenter, Mechanic, Cleaning Service, Appliance Repair, Mobile Repair, Computer Technician, and more.
- **Provider Metadata:** To ensure high-quality matching, the dataset incorporates geo-coordinates, availability status, experience levels, completed jobs, response times, verification status, languages supported, contact details, pricing tiers, and ratings.

3. Development Methodology & Context Engineering

To ensure rapid but stable development, the backend team will strictly utilize Google Antigravity's Spec-Driven Development (SDD) workflows.

- **Context Engineering:** We are shifting our focus from basic prompt engineering to system-level Context Engineering. This means systematically designing the entire information flow—including instructions, conversation history, retrieved data, and tool usage—to ensure the agents have the precise constraints needed to operate without hallucinations.
- **Plan-then-Implement Loop:** Before generating code, Antigravity will be forced into planning mode to output structured artifacts for architectural review.

4. Core Orchestration: Google ADK V2 & Human Handoff

The backend logic will be decoupled into specialized agents using Google ADK V2.

- **Model Context Protocol (MCP) & A2A:** We will expose our databases to the agents using MCP servers, and utilize the Agent-to-Agent (A2A) protocol for secure communication across distributed ADK agents.
- **Human Handoff Logic:** Recognizing that autonomous systems require supervision, the architecture will include structured transfers of control. We will implement escalation logic and persistent context sharing to allow a seamless, low-friction handoff to human operators for unexpected failures or edge cases, ensuring work resumes cleanly without losing session data.

5. The Hybrid AI Knowledge Engine

We will implement a 4-tier hybrid knowledge architecture powered by state-of-the-art open-source technology:

- **Foundational Model (Gemma 4):** We will transition to the Gemma 4 open-source LLM family, which is purpose-built for advanced reasoning and agentic workflows, offering native support for function calling and multi-step planning.
- **Generation & Streaming Configuration:** To guarantee a responsive user experience, the model's generation configuration will leverage optimized top-p and top-k sampling parameters for deterministic reasoning. Output will be delivered via streaming with delta updates for rapid, real-time responses to the frontend.
- **Embedding Layer:** We will utilize the BAAI/bge-large-en-v1.5 embedding model. This high-performance model translates English and Roman Urdu code-switched text into precise 1024-dimensional vector representations, delivering high-fidelity semantic embeddings for similarity searches.
- **Agentic RAG & Vector Store (Weaviate):** Highly dynamic data (live provider availability) will be queried through an Agentic RAG pipeline. The semantic search will be powered by Weaviate as our primary vector store, ensuring rapid similarity matching for the 1024-dimensional vectors.
- **Cache-Augmented Generation (CAG) via Rust:** Static "Golden Knowledge" will be preloaded into the LLM's extended context window. To guarantee low latency and memory safety, the CAG KV-cache operations will be implemented in Rust via the PyO3 binding library.
- **Fine-Tuning:** Parameter-efficient fine-tuning on Gemma 4 will handle the nuances of the informal economy terminology.

6. State Management & Observability

- **Redis & PostgreSQL:** Redis will act as the ultra-fast caching layer for short-term session states, while PostgreSQL will handle episodic memory and ACID guarantees for stateful workflows.
- **OpenTelemetry (Tracing & Spanning):** To maintain observability over the agentic pipeline, we will integrate OpenTelemetry. This will allow us to capture distributed traces and spans across the entire agent-to-backend interaction chain—mapping out prompt assembly, Weaviate vector retrievals, and MCP tool execution—and export them to our observability dashboard for performance monitoring.

7. Cloud Deployment Environment

The entire backend will be packaged in Docker containers and deployed serverlessly on GCP (Google Cloud Run), securely connecting to our PostgreSQL, Redis, and Weaviate clusters within a private network boundary.